

Proyecto Ordinario: Sistemas Distribuidos y Alta Disponibilidad

Fecha límite de entrega y demostración: 26 de junio de 2026

Modalidad: Trabajo colaborativo (Parejas)

Valor de evaluación: 50% de la calificación final

Objetivo del Proyecto

Durante las sesiones de laboratorio, se implementó de manera exitosa una infraestructura de orquestación basada en Kubernetes (MicroShift). El objetivo central de esta evaluación es aplicar dichas competencias para el despliegue de un sistema distribuido funcional.

Los estudiantes deberán desarrollar un sistema mediante tecnologías web y orquestarlo en su clúster local, garantizando la **Alta Disponibilidad (HA)**. La rúbrica priorizará una arquitectura sólida, resiliente y tolerante a fallos sobre la complejidad de la lógica de negocio del aplicativo.

Prerrequisitos de Evaluación: Auditoría del Clúster

Antes de proceder con el desarrollo y despliegue del proyecto, es indispensable verificar la integridad y operatividad del clúster de forma local (sin requerimiento de conexión a internet externa):

1. **Verificación de Estado del Nodo:** Ejecutar el comando `oc get nodes`.
 - *Criterio de validación:* En la columna `STATUS` se debe visualizar el estado **Ready**. En caso de mostrar **NotReady** o un error de conexión, será necesario revisar la configuración del Firewall y verificar la ejecución del servicio (`sudo systemctl status microshift`).
2. **Verificación de Cargas de Trabajo:** Ejecutar el comando `oc get pods -A`.
 - *Criterio de validación:* Se presentará la lista de procesos internos del orquestador. La mayoría de estos deberán encontrarse en estado **Running** o **Completed**.

Nota: La superación de ambas pruebas técnicas confirma que la infraestructura se encuentra en condiciones óptimas para el despliegue del sistema.

Metodología de Trabajo Colaborativo

Para optimizar la eficiencia, se sugiere la siguiente asignación de responsabilidades:

- **Rol de Desarrollo (Software):** Responsable de la programación lógica del cliente (interfaces web en HTML/JS) y del servidor backend (Node.js, Python, Java, etc.).
- **Rol de Arquitectura (Infraestructura):** Responsable de la contenerización del aplicativo mediante Podman, la elaboración del manifiesto declarativo (YAML) y el despliegue efectivo en MicroShift, gestionando la correcta exposición de los puertos de red.

Arquitectura Requerida (Modelo de 3 Nodos)

El sistema debe cumplir estrictamente con la rúbrica de "comunicación en red entre al menos 3 nodos". La topología esperada es la siguiente:

1. **Nodo 1 (Cliente Terminal):** Interfaz gráfica (página web) ejecutada desde un dispositivo terminal (equipo de cómputo o dispositivo móvil).
2. **Nodo 2 (Servidor - Réplica Principal):** Instancia primaria del servicio backend, orquestada como un Pod dentro de MicroShift.
3. **Nodo 3 (Servidor - Redundancia):** Segunda instancia del backend para asegurar la Alta Disponibilidad; alternatively, un contenedor dedicado exclusivamente a la persistencia de datos (base de datos ligera).

Opciones de Implementación

Los equipos deberán seleccionar **una** de las siguientes alternativas, enfocándose en la eficiencia del modelo distribuido por encima de la complejidad funcional:

1. **Sistema de Comunicación Concurrente (Chat):** Implementación de un servidor web empleando WebSockets (ej. Socket.io) que permita la transmisión de mensajes en tiempo real entre múltiples clientes conectados.
2. **Aplicación Lúdica Distribuida (Lotería/Gato):** Servidor centralizado que gestione la lógica, los estados concurrentes y determine el resultado de partidas entre clientes interactivos.
3. **API REST de Gestión de Datos:** Arquitectura de microservicio orientada a transacciones que procese peticiones HTTP (**GET**, **POST**) para el registro y consulta de información estructurada mediante formato JSON.

Evaluación de Tolerancia a Fallos (Demostración Práctica)

El 26 de junio se llevará a cabo la auditoría del clúster y del aplicativo. El criterio determinante será la **Tolerancia a Fallos (Chaos Testing)**.

Durante la operación normal del sistema desde las terminales cliente, el docente instruirá la eliminación manual de uno de los Pods del servidor (**`oc delete pod [nombre]`**).

- **Criterio de Éxito:** El plano de control de MicroShift deberá detectar la anomalía y aprovisionar una nueva réplica en cuestión de milisegundos. El servicio deberá mantener su continuidad operativa sin interrupciones perceptibles para el usuario final.
- **Nota:** La superación de esta prueba representa el 20% de la calificación total del proyecto.

Entregables y Ponderación

Para obtener el 100% de la calificación estipulada, se deberá presentar lo siguiente:

1. **[10 pts] Diseño y Arquitectura:** Diagramas formales de componentes y de secuencia que ilustren el flujo de datos entre el cliente y el clúster. (Se incluye en la documentación)
2. **[10 pts] Protocolo de Comunicación:** Código fuente que demuestre la implementación exitosa de WebSockets, servicios REST o RPC, y serialización de datos en formato JSON.
3. **[10 pts] Infraestructura y Despliegue:** Sistema operando funcionalmente dentro de la máquina virtual, soportado en contenedores y orquestado en MicroShift.
4. **[10 pts] Tolerancia a Fallos:** Finalización exitosa de la demostración práctica de recuperación de servicios en vivo.
5. **[10 pts] Documentación Técnica:** Código fuente debidamente comentado (se muestra en el laboratorio) y un manual de despliegue breve, que incluya y explique el archivo **YAML**. (Se incluye en la documentación)